

POSTECH



TECH CHALLENGE

Tech Challenge é o projeto da fase que englobará os conhecimentos obtidos em todas as disciplinas da fase. Esta é uma atividade que, a princípio, deve ser desenvolvida em grupo. É importante atentar-se ao prazo de entrega, pois trata-se de uma atividade obrigatória, uma vez que vale 90% da nota de todas as disciplinas da fase.

- Atividade em grupo ou individual · Obrigatória · Avaliada.
- Entrega obrigatória: Repositório GitHub + Vídeo de 5 minutos (método STAR).
- Entrega opcional: Deploy em ambiente de produção em nuvem (AWS, Azure ou GCP).
- Tema Central: Pipeline Preditivo de Churn com Modelagem Clássica e API REST

Contexto

Uma operadora de telecomunicações está perdendo clientes em ritmo acelerado. A diretoria precisa de um modelo preditivo de churn que classifique clientes com risco de cancelamento. Assim, o grupo deve construir o projeto desde a análise exploratória até o modelo servido via API, aplicando as boas práticas de engenharia de software aprendidas na Fase 1.

O modelo central de entrega será desenvolvido utilizando o ecossistema do **Scikit-Learn** (comparando modelos lineares, baseados em árvores e redes neurais simples), empacotado em uma arquitetura modular e servido utilizando o framework **FastAPI**.

Requisitos Obrigatórios

Repositório GitHub

- Estrutura organizada: src/, data/, models/, tests/, notebooks/, docs/.

- README.md completo com instruções de setup, execução e descrição do projeto.
- Gerenciamento de dependências via pyproject.toml ou requirements.txt.
- Histórico de commits limpo e significativo evidenciando o trabalho em grupo.
- .gitignore adequado para projetos de Python e Machine Learning.

Vídeo (5 minutos — método STAR)

- **Situation:** Qual o problema de negócio e o contexto do dataset?
- **Task:** Qual a tarefa do grupo e os objetivos técnicos?
- **Action:** Quais decisões técnicas foram tomadas (limpeza de dados, escolha do modelo final, construção da API)?
- **Result:** Quais os resultados obtidos com o modelo e a API funcionando?

Bibliotecas Requeridas

- **Scikit-Learn** — pipelines de pré-processamento, baselines, árvores de decisão e redes neurais (MLPClassifier).
- **FastAPI** — API de inferência do modelo.
- **Pytest** — para testes básicos do código.

Boas Práticas Obrigatórias

- Separação clara entre experimentação (Notebooks) e código produtivo (módulos em .py).
- Seeds fixados para reprodutibilidade.
- Pelo menos dois testes automatizados simples (ex.: teste de uma função de pré-processamento e teste de status da API).
- Documentação básica de limitações do modelo (Model Card).

Etapas de Desenvolvimento (4 Etapas)

Etapas 1 — Entendimento e Preparação (Disciplinas: Ciclo de Vida e Fundamentos)

Foco: formulação do problema, exploração de dados e construção de baselines.

- Tarefas:
 - Preencher o ML Canvas (stakeholders, métricas de negócio).
 - Análise Exploratória de Dados (EDA): volume, qualidade e distribuição das features.
 - Definir métrica técnica (ex.: F1, AUC-ROC) e métrica de negócio.
 - Treinar modelo baseline com Regressão Logística (Scikit-Learn).
- **Entregável:** Notebook de EDA + Baseline criado e documentado.

Etapas 2 — Modelagem e Avaliação (Disciplina: Fundamentos de Modelos de ML)

Foco: Treinamento, comparação e avaliação de modelos.

- Tarefas:
 - Treinar um modelo baseado em Árvores de Decisão ou Ensembles (ex.: Random Forest).
 - Treinar uma Rede Neural simples utilizando o MLPClassifier do Scikit-Learn.
 - Aplicar Validação Cruzada para garantir a robustez.
 - Comparar o desempenho dos modelos (Linear, Árvore e MLP) usando as métricas definidas e escolher um modelo "campeão".
 - Registrar os resultados dos experimentos (pode ser em planilhas, logs em texto ou ferramentas de tracking opcionais).
- **Entregável:** Tabela comparativa de modelos + Modelo final escolhido e salvo (.pkl ou .joblib).

Etapa 3 — Engenharia de Software e API (Disciplinas: Eng. de Software e APIs para Inferência)

Foco: refatoração profissional, testes e API de inferência.

- Tarefas:
 - Refatorar o código de pré-processamento e predição do modelo campeão para módulos Python dentro da pasta `src/` (usando boas práticas e funções claras).
 - Escrever testes unitários com `pytest` para garantir que as funções principais rodem sem erro.
 - Construir a API utilizando **FastAPI** com dois endpoints: `/health` (para checar se a API está no ar) e `/predict` (para receber os dados do cliente e retornar a propensão de churn).
- **Entregável:** Repositório refatorado + API funcional testada.

Etapa 4 — Documentação e Entrega Final (Todas as disciplinas)

Foco: consolidação, documentação e vídeo de apresentação.

- Tarefas:
 - Gerar um Model Card simples documentando performance, limitações e possíveis vieses do modelo treinado.
 - Finalizar o `README.md` ensinando como instalar as dependências e rodar a API localmente.
 - Gravar um vídeo de 5 min (método STAR) demonstrando o projeto.
- **Entregável:** Repositório final validado + vídeo STAR.

Critérios de Avaliação

Critério	Peso	Descrição
Qualidade do Código e Engenharia	25%	Organização das pastas, modularidade, separação de responsabilidades e código limpo.
Modelagem de Machine Learning	25%	Qualidade da EDA, treinamento correto dos modelos no Scikit-Learn e coerência na comparação do modelo campeão vs baseline.
API de Inferência	20%	FastAPI funcional, recebendo chamadas e retornando previsões corretamente.
Testes e Reprodutibilidade	10%	Uso correto do pyproject.toml/requirements.txt, seeds fixados e testes do pytest passando.
Documentação e Vídeo STAR	20%	README claro, Model Card preenchido e clareza na explicação em vídeo (≤ 5 min).

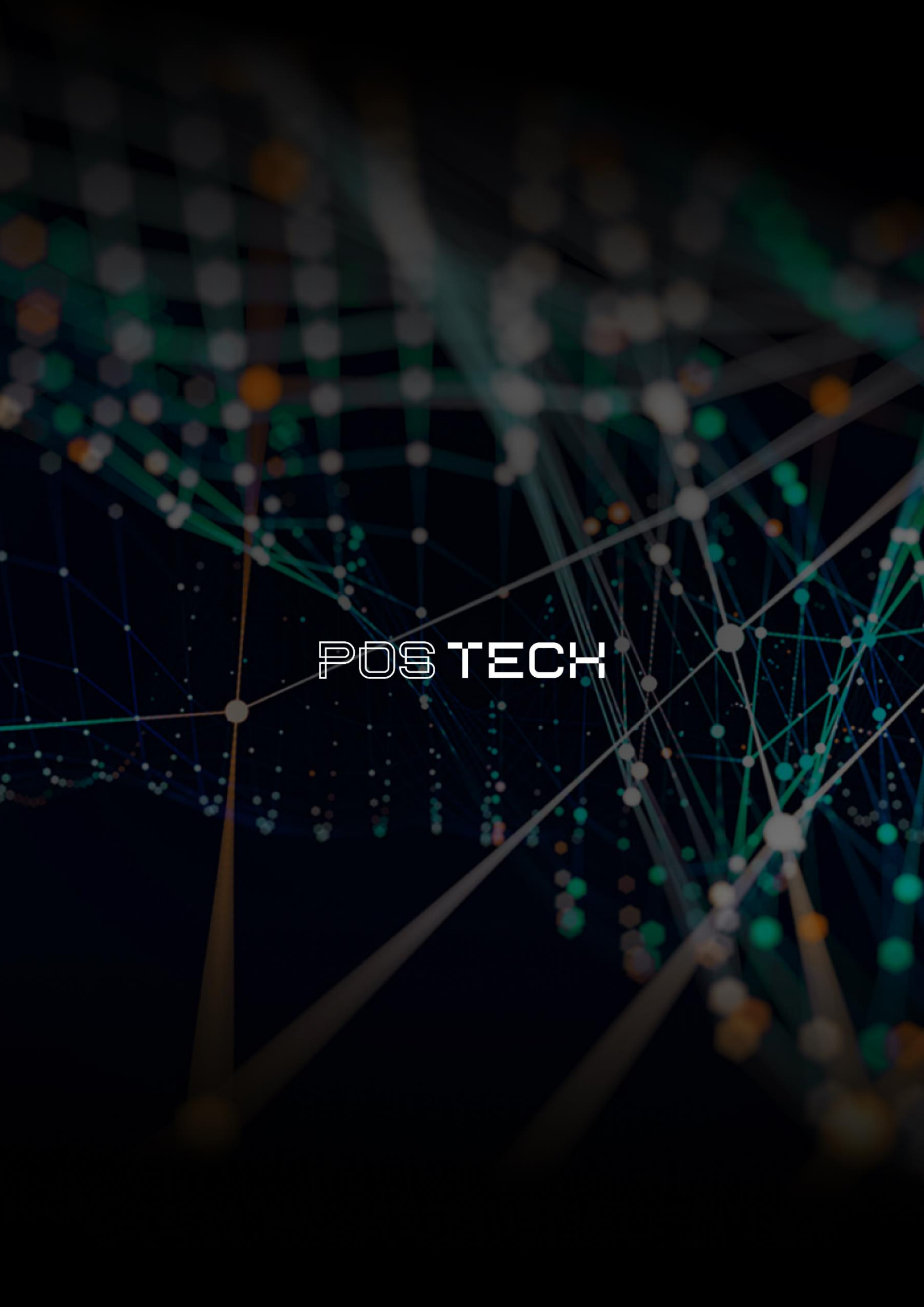
Dataset Sugerido

Dataset público de telecomunicações com variáveis tabulares (ex.: **Telco Customer Churn — IBM** do Kaggle). Alternativas: qualquer dataset de classificação binária focado em comportamento de clientes com > 5.000 registros.

Passo a Passo Resumido

- **[Etapa 1]** EDA + Definição de Métricas + Baseline de Regressão Logística.
- **[Etapa 2]** Treino de Random Forest e MLP (Scikit-Learn) + Comparação de Modelos.
- **[Etapa 3]** Refatoração para a pasta src/ + Testes Pytest + Criação da API FastAPI.
- **[Etapa 4]** Finalização do README + Model Card + Gravação do Vídeo STAR.

Caso tenha qualquer dúvida, não deixe de nos procurar no Discord!

The background is a dark, abstract network visualization. It features a complex web of thin, light-colored lines connecting numerous small, glowing nodes. The nodes are primarily white and light blue, with some larger, more prominent nodes in green and orange. The overall effect is a sense of dynamic connectivity and data flow, typical of a digital or technological theme.

POSTECH